AIAC Assignment-14.5

Name : G.Samshray

Ht-no: 2303A51670

Batch :23

Task 1 – Image Gallery Website

Create an image gallery web page.

Requirements:

• Display images in a grid layout.

• Use AI to:

o Optimize layout using CSS Grid.

o Add hover zoom or overlay effects.

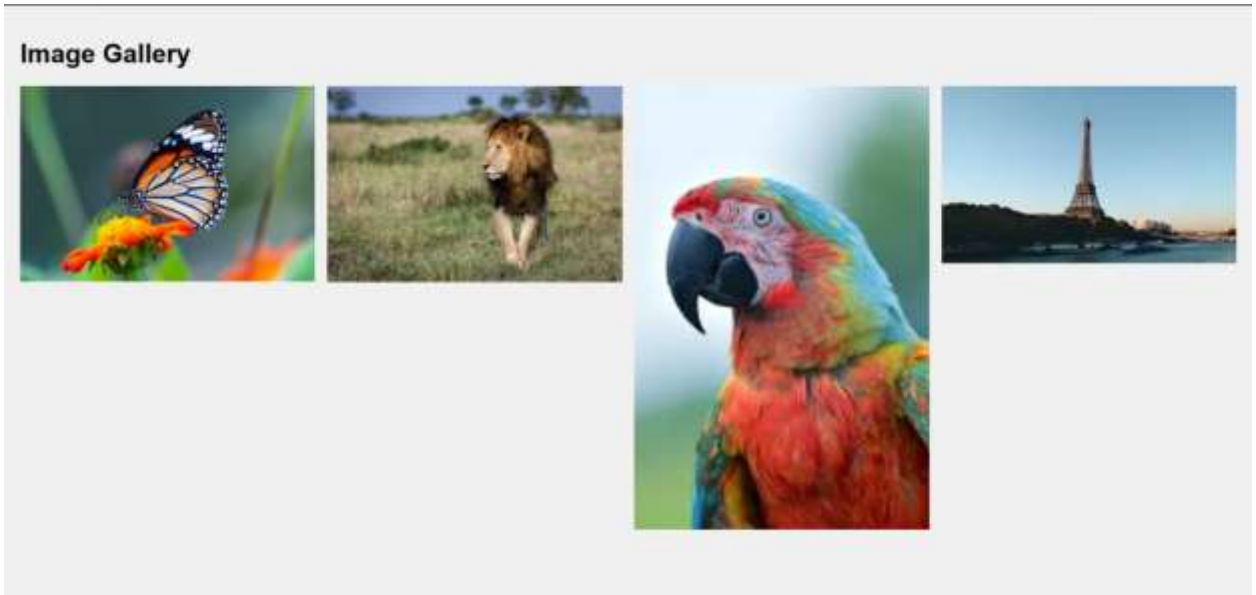
o Ensure mobile responsiveness.

Code:

```
<!--Create an image gallery web page.
Display images in a grid layout.
Optimize layout using CSS Grid.
Add hover zoom or overlay effects.
Ensure mobile responsiveness.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Gallery</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 20px;
            background-color: #f0f0f0;
        }
        .gallery {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
            gap: 15px;
        }
        .gallery-item {
            position: relative;
            overflow: hidden;
        }
        .gallery-item img {
            width: 100%;
            height: auto;
            transition: transform 0.3s ease;
        }
        .gallery-item:hover img {
            transform: scale(1.1);
        }
        @media (max-width: 600px) {
            .gallery {
                grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
            }
        }
    </style>
</head>
<body>
```

```html
    <h1>Image Gallery</h1>
    <div class="gallery">
        <div class="gallery-item">
            <img src="C:\Users\PC\Desktop\Image.jpg" alt="Image 1">
        </div>
        <div class="gallery-item">
            <img src="C:\Users\PC\Downloads\download.jpg" alt="Image 2">
        </div>
        <div class="gallery-item">
            <img src="C:\Users\PC\Downloads\download (1).jpg" alt="Image 3">
        </div>
        <div class="gallery-item">
            <img src="C:\Users\PC\Downloads\download (2).jpg" alt="Image 4">
        </div>
        <!-- Add more images as needed -->
    </div>
</body>
</html>
```

Output:



Task 2 – Profile Card Generator

Create a profile card web page.

Requirements:

• Display user image, name, role, and social media links.

• Use AI to:

o Suggest modern UI colors and fonts.

o Design the card using Flexbox.

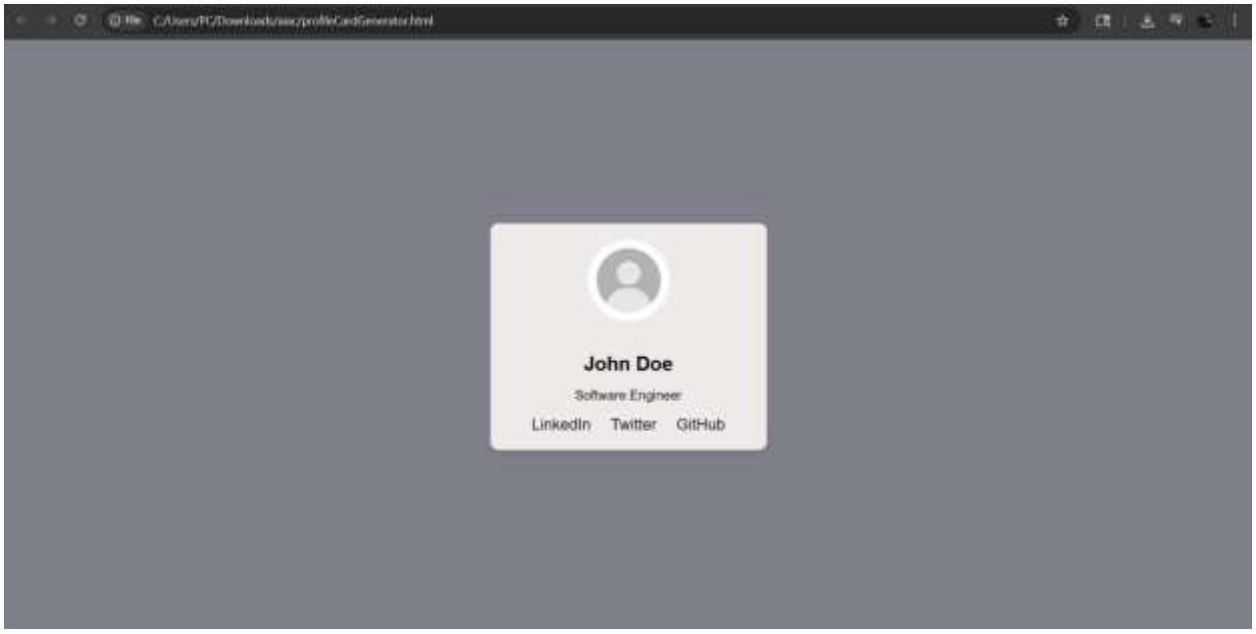
o Add hover animation and shadow effects

Code:

```
C: > Users > PC > Downloads > aiac > profileCardGenerator.html > html > head > style > .profile-card:hover
<!--Create a profile card web page.
Display user image, name, role, and social media links.
Suggest modern UI colors and fonts.
Design the card using flexbox.
Add hover animation and shadow effects-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Profile Card</title>
    <style>
        body {
            font-family: 'Arial', sans-serif;
            background-color: #7f7f8c;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }
        .profile-card {
            background-color: #efebeb;
            border-radius: 10px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            width: 300px;
            text-align: center;
            padding: 20px;
            transition: transform 0.3s ease, box-shadow 0.3s ease;
        }
        .profile-card:hover {
            transform: translateY(-10px);
            box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
        }
        .profile-image {
            width: 100px;
            height: 100px;
```

```
            object-fit: cover;
            margin-bottom: 15px;
        }
        .profile-name {
            font-size: 1.5em;
            margin-bottom: 5px;
        }
        .profile-role {
            color: #0e0b0b;
            margin-bottom: 15px;
        }
        .social-links a {
            color: #100a0e;
            text-decoration: none;
            margin: 0 10px;
            font-size: 1.2em;
        }
        .social-links a:hover {
            color: #000;
        }
    </style>
</head>
<body>
    <div class="profile-card">
        <img src="C:\Users\PC\Downloads\download.png" alt="Profile Image" class="profile-image">
        <h2 class="profile-name">John Doe</h2>
        <p class="profile-role">Software Engineer</p>
        <div class="social-links">
            <a href="#" target="_blank">LinkedIn</a>
            <a href="#" target="_blank">Twitter</a>
            <a href="#" target="_blank">GitHub</a>
        </div>
    </div>
</body>
</html>
```

Output:



Task 3 -Develop a dynamic shopping cart system.

Requirements:

• Add, remove, and update product quantities.

• Display real-time total price.

• Use AI to:

o Manage cart state using JavaScript objects.

o Persist cart data in localStorage.

Code:

```
C: > Users > PC > Downloads > aac > DynamicShoppingCart.html > html
<!--Develop a dynamic shopping cart system.
Add, remove, and update product quantities.
Display real-time total price.
Manage cart state using JavaScript objects.
Persist cart data in localStorage.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic Shopping Cart</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .product {
            display: flex;
            justify-content: space-between;
            margin-bottom: 10px;
        }
        .cart {
            margin-top: 20px;
        }
        .cart-item {
            display: flex;
            justify-content: space-between;
            margin-bottom: 10px;
        }
        .total {
            font-weight: bold;
        }
    </style>
</head>
<body>
    <h1>Dynamic Shopping Cart</h1>
    <div id="products">
        <div class="product">
            <span>Product 1 - $10</span>
            <button onclick="addToCart('Product 1', 10)">Add to Cart</button>
        </div>
        <div class="product">
            <span>Product 2 - $20</span>
            <button onclick="addToCart('Product 2', 20)">Add to Cart</button>
```

```
        </div>
        <div class="product">
            <span>Product 3 - $30</span>
            <button onclick="addToCart('Product 3', 30)">Add to Cart</button>
        </div>
    </div>

    <div class="cart">
        <h2>Shopping Cart</h2>
        <div id="cart-items"></div>
        <div class="total">Total: $<span id="total-price">0</span></div>
    </div>

    <script>
        let cart = JSON.parse(localStorage.getItem('cart')) || {};

        function addToCart(product, price) {
            if (cart[product]) {
                cart[product].quantity += 1;
            } else {
                cart[product] = { price: price, quantity: 1 };
            }
            updateCart();
        }

        function removeFromCart(product) {
            delete cart[product];
            updateCart();
        }

        function updateCart() {
            const cartItems = document.getElementById('cart-items');
            cartItems.innerHTML = '';
            let totalPrice = 0;

            for (let product in cart) {
                const item = cart[product];
                totalPrice += item.price * item.quantity;
                cartItems.innerHTML += `
                    <div class="cart-item">
                        <span>${product} - $${item.price} x ${item.quantity}</span>
                        <button onclick="removeFromCart('${product}')">Remove</button>
```

```
                    </div>
                `;
            }

            document.getElementById('total-price').textContent = totalPrice.toFixed(2);
            localStorage.setItem('cart', JSON.stringify(cart));
        }

        updateCart();
    </script>
</body>
</html>
```

Output:

Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

• Admin, Editor, User roles.

• Conditional UI rendering.

• Use AI to:

o Design access control logic.

o Maintain secure state transitions.

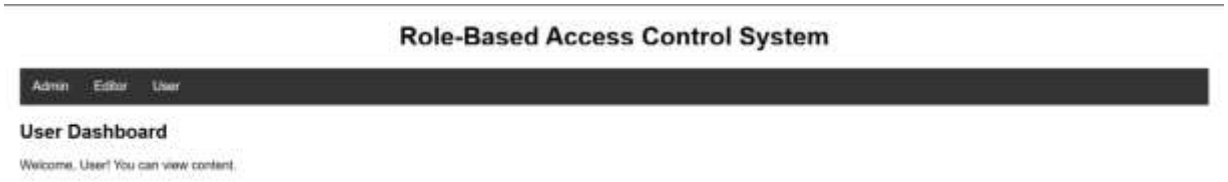
o Create scalable UI components.

Code:

```
C: > Users > PC > Downloads > aac > roleBasedAccessControl.html > html > body > div.navbar > a
<!-- Role-Based Access Control System
 Create a role-based web application.
Admin, Editor, User roles.
Conditional UI rendering.

 Design access control logic.
 Maintain secure state transitions.
  Create scalable UI components.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Role-Based Access Control</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .navbar {
            background-color: #333;
            overflow: hidden;
        }
        .navbar a {
            float: left;
            display: block;
            color: white;
            text-align: center;
            padding: 14px 16px;
            text-decoration: none;
        }
        .navbar a:hover {
            background-color: #ddd;
            color: black;
        }
        .content {
            margin-top: 20px;
        }
    </style>
</head>
<body>
    <h1 style="text-align: center;">Role-Based Access Control System</h1>
    <div class="navbar">
        <a href="#" onclick="setRole('Admin')">Admin</a>
```

```
        <a href="#" onclick="setRole('Editor')">Editor</a>
        <a href="#" onclick="setRole('User')">User</a>
    </div>

    <div class="content" id="content">
        <!-- Content will be rendered based on role -->
    </div>

    <script>
        let currentRole = 'User'; // Default role

        function setRole(role) {
            currentRole = role;
            updateContent();
        }

        function updateContent() {
            const contentDiv = document.getElementById('content');
            contentDiv.innerHTML = ''; // Clear existing content

            if (currentRole === 'Admin') {
                contentDiv.innerHTML = '<h2>Admin Dashboard</h2><p>Welcome, Admin! You have full access.</p>';
            } else if (currentRole === 'Editor') {
                contentDiv.innerHTML = '<h2>Editor Dashboard</h2><p>Welcome, Editor! You can edit content.</p>';
            } else if (currentRole === 'User') {
                contentDiv.innerHTML = '<h2>User Dashboard</h2><p>Welcome, User! You can view content.</p>';
            }
        }

        // Initialize content based on default role
         updateContent();

    </script>
</body>
</html>
```

Output:



Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

• Switch languages dynamically.

• Persist language preference.

• Use AI to:

o Structure translation files.

o Handle RTL/LTR layouts.

o Improve accessibility.

Code:

```
C: > Users > PC > Downloads > aaa > multiLanguageWebsite.html > html > body
<!--
Design a multilingual website.
Switch languages dynamically.
Persist language preference.

Structure translation files.
Handle RTL/LTR layouts.
Improve accessibility.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Multilingual Website</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .language-switcher {
            margin-bottom: 20px;
        }
        .content {
            font-size: 1.2em;
        }
        .background {
            background-color: #8e8686;
            padding: 20px;
            border-radius: 10px;
        }
    </style>
</head>
<body>
    <h1 style="text-align: center;">Multilingual Website</h1>
    <div class="language-switcher">
        <button onclick="setLanguage('en')">English</button>
        <button onclick="setLanguage('es')">Spanish</button>
        <button onclick="setLanguage('fr')">French</button>
    </div>

    <div class="content" id="content">
        <!-- Content will be rendered based on language -->
    </div>

```

```
    <script>
        const translations = {
            en: {
                welcome: "Welcome to our multilingual website!",
                description: "This website supports multiple languages. Please select your preferred language."
            },
            es: {
                welcome: "¡Bienvenido a nuestro sitio web multilingüe!",
                description: "Este sitio web admite varios idiomas. Por favor, seleccione su idioma preferido."
            },
            fr: {
                welcome: "Bienvenue sur notre site web multilingue!",
                description: "Ce site web prend en charge plusieurs langues. Veuillez sélectionner votre langue préférée."
            }
        };

        let currentLanguage = 'en'; // Default language

        function setLanguage(lang) {
            currentLanguage = lang;
            localStorage.setItem('preferredLanguage', lang);
            updateContent();
        }

        function updateContent() {
            const content = translations[currentLanguage];
            document.getElementById('content').innerHTML = `
                <h2>${content.welcome}</h2>
                <p>${content.description}</p>
            `;
        }

        // Load preferred language on page load
        window.onload = function() {
            const savedLanguage = localStorage.getItem('preferredLanguage');
            if (savedLanguage) {
                setLanguage(savedLanguage);
            } else {
                updateContent();
            }
        };
        </script>
</body>
```

Output:

Task 6 – Online Examination System

Create an online exam interface.

Requirements:

• Timer, questions, and submission.

• Prevent multiple submissions.

• Use AI to:

o Manage exam state.

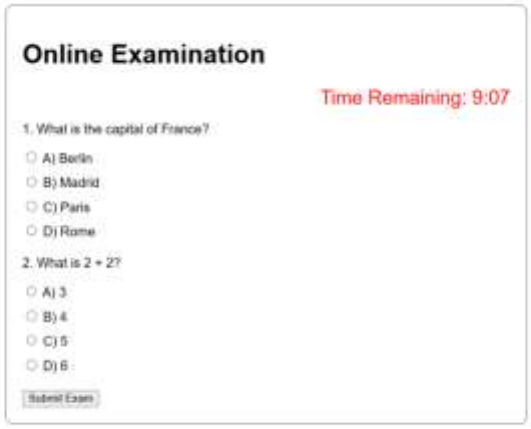
o Improve exam UX.

o Ensure accessibility.

Code:

```
C: > Users > PC > Downloads > aac > onlineExaminationSystem.html > html > body
<!--
Create an online exam interface.

Timer, questions, and submission.
Prevent multiple submissions.
Manage exam state.
Improve exam UX.
Ensure accessibility.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Examination System</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .exam-container {
            max-width: 600px;
            margin: auto;
            padding: 20px;
            border: 1px solid #888343;
            border-radius: 10px;
        }
        .question {
            margin-bottom: 20px;
        }
        .options {
            list-style-type: none;
            padding: 0;
        }
        .options li {
            margin-bottom: 10px;
        }
        .timer {
            font-size: 1.5em;
            color: red;
            text-align: right;
        }
    </style>
</head>
<body>
```

```
    <div class="exam-container">
        <h1>Online Examination</h1>
        <div class="timer" id="timer">Time Remaining: 10:00</div>
        <form id="exam-form">
            <div class="question">
                <p>1. What is the capital of France?</p>
                <ul class="options">
                    <li><input type="radio" name="q1" value="A"> A) Berlin</li>
                    <li><input type="radio" name="q1" value="B"> B) Madrid</li>
                    <li><input type="radio" name="q1" value="C"> C) Paris</li>
                    <li><input type="radio" name="q1" value="D"> D) Rome</li>
                </ul>
            </div>
            <div class="question">
                <p>2. What is 2 + 2?</p>
                <ul class="options">
                    <li><input type="radio" name="q2" value="A"> A) 3</li>
                    <li><input type="radio" name="q2" value="B"> B) 4</li>
                    <li><input type="radio" name="q2" value="C"> C) 5</li>
                    <li><input type="radio" name="q2" value="D"> D) 6</li>
                </ul>
            </div>
            <button type="submit">Submit Exam</button>
        </form>
    </div>
    <script>
        let timeRemaining = 600; // 10 minutes in seconds
        const timerElement = document.getElementById('timer');
        const examForm = document.getElementById('exam-form');
        let examSubmitted = false;

        const timerInterval = setInterval(() => {
            if (timeRemaining > 0) {
                timeRemaining--;
                const minutes = Math.floor(timeRemaining / 60);
                const seconds = timeRemaining % 60;
                timerElement.textContent = `Time Remaining: ${minutes}:${seconds < 10 ? '0' : ''}${seconds}`;
            } else {
                clearInterval(timerInterval);
                if (!examSubmitted) {
                    alert('Time is up! Your exam will be submitted automatically.');
                    submitExam();
                }
```

```
        }
    }, 1000);

    examForm.addEventListener('submit', (e) => {
        e.preventDefault();
        if (!examSubmitted) {
            submitExam();
        } else {
            alert('You have already submitted the exam.');
        }
    });

    function submitExam() {
        examSubmitted = true;
        clearInterval(timerInterval);
        // Here you would typically send the answers to the server
        alert('Exam submitted successfully!');
    }
    window.onload = function() {
        const savedState = localStorage.getItem('examState');
        if (savedState) {
            const examState = JSON.parse(savedState);
            timeRemaining = examState.timeRemaining;
            examSubmitted = examState.examSubmitted;
            updateTimer();
        }
    };
    window.onbeforeunload = function() {
        const examState = {
            timeRemaining: timeRemaining,
            examSubmitted: examSubmitted
        };
        localStorage.setItem('examState', JSON.stringify(examState));
    };
    function updateTimer() {
        const minutes = Math.floor(timeRemaining / 60);
        const seconds = timeRemaining % 60;
        timerElement.textContent = `Time Remaining: ${minutes}:${seconds < 10 ? '0' : ''}${seconds}`;
    }
    </script>
```

Output: